SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:25**

ANNEXURE A
APPLICATION BASED QUESTIONS

Code of Conduct:

I T.KAVYA(192411257) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Co1 ASSESSMENT TOOL-1

Name: T. Kavya

Reg No: 192411257

Course code: CSA1221

Course name: COMPUTER ARCHITECTURE

Q1. Smart City Surveillance System – CPU, Memory and I/O Interaction

Introduction

A smart city surveillance system continuously captures and processes video streams from hundreds of cameras to detect accidents, crimes, traffic violations, and suspicious activities. The system consists of **Input/Output (I/O) devices**, **main memory (RAM)**, and the **Central Processing Unit (CPU)**. These components work together to process enormous amounts of real-time data.

During peak traffic hours, the volume of video data increases significantly. If the CPU, memory, and I/O devices are unable to communicate efficiently, the system experiences **high latency**, delayed threat detection, and reduced overall performance.

Therefore, understanding the interaction between CPU, memory, and I/O devices is essential for improving system efficiency.

---

1. Interaction Between CPU, Memory, and I/O

The three major components perform different tasks but depend on each other.

CPU (Central Processing Unit)

- Executes program instructions.
- Processes video frames.
- Detects objects using AI algorithms.
- Makes decisions and sends alerts.

Memory (RAM)

- Temporarily stores incoming video frames.
- Stores instructions currently executed by the CPU.
- Provides high-speed access to data.

Input/Output Devices

- CCTV cameras capture live video.
- Network interface transfers video data.
- Storage devices save recordings.

Working Process

CCTV Cameras

Input/Output Controller

Main Memory (RAM)

CPU

Threat Detection

Display / Alarm / Storage

Explanation

1. Cameras continuously capture video.
2. Video is transferred through I/O devices.
3. Data is stored temporarily in RAM.
4. CPU fetches the video frames.
5. CPU processes each frame.
6. Results are displayed or stored.

If any component becomes slow, the entire system performance decreases.

---

2. Causes of Lag During Peak Hours

During peak hours:

- More cameras become active.
- Higher-resolution videos are received.
- More AI algorithms execute simultaneously.
- CPU receives excessive interrupts.
- Memory bandwidth becomes insufficient.

As a result,

- CPU waits for memory.
- Memory waits for I/O.
- I/O devices wait for bus availability.

This increases response time.

Major Bottlenecks

Component	Problem	Effect
CPU	Too many instructions	High processing delay
Memory	Insufficient bandwidth	CPU waits for data
I/O Devices	Continuous data transfer	Queue formation
Bus	Shared communication path	Data congestion
Cache	Frequent cache misses	Increased memory access time

---

3. Contribution of CPU, Memory, and I/O to Performance Issues

CPU Bottleneck

The CPU executes millions of instructions every second.

For every video frame it performs:

- Image decoding
- Motion detection
- Object recognition
- Threat classification

When the number of frames increases,

- Instruction queue becomes full.
- CPI (Clock Cycles Per Instruction) increases.
- CPU utilization reaches nearly 100%.

This results in delayed processing.

Memory Bottleneck

The CPU cannot process data unless it is available in RAM.

Large video frames occupy significant memory.

Example:

One Full HD frame ≈ 6 MB

If 200 cameras send:

200 × 6 MB = 1200 MB per frame

Huge memory traffic causes:

- Frequent memory access
- Cache misses
- Increased waiting time

I/O Bottleneck

Cameras continuously generate data.

If I/O controllers cannot transfer data quickly,

- Buffers become full.
- Frames are dropped.
- CPU waits for new data.

Consequently,

Threat detection becomes slower.

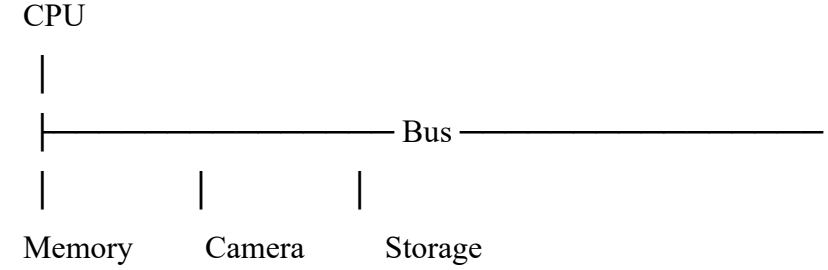
---

4. Role of Bus Structure

The bus is the communication pathway connecting CPU, memory, and I/O devices.

Its performance directly affects the surveillance system.

Single Bus Architecture



Working

All devices use the same communication channel.

Only one transfer occurs at a time.

Advantages

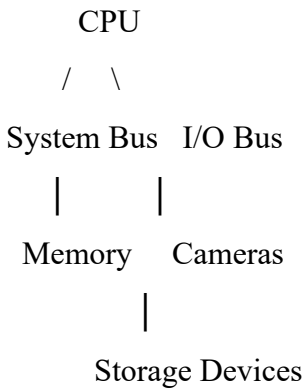
- Simple design
- Low cost
- Easy implementation

Disadvantages

- Bus congestion
- CPU waits for data transfer
- Slow communication
- Reduced throughput

During peak hours, this architecture causes severe delays because many cameras attempt to access the bus simultaneously.

Multi-Bus Architecture



Different buses handle different operations simultaneously.

Example:

- CPU accesses RAM.
- Cameras send video.
- Storage writes recordings.

All occur simultaneously.

Comparison of Single Bus and Multi-Bus

Feature	Single Bus	Multi Bus
Number of buses	One	Multiple
Data transfer	One at a time	Parallel transfers

Feature	Single Bus	Multi Bus
Performance	Low	High
Bus congestion	High	Very low
CPU waiting time	High	Low
Cost	Low	Higher
Scalability	Limited	Excellent
Suitable for surveillance	No	Yes

---

5. Instruction Execution Cycle

The CPU executes every instruction using the instruction cycle.

The stages are:

Fetch

↓

Decode

↓

Execute

↓

Store Result

Fetch

Instruction is fetched from memory.

Decode

Control Unit identifies the operation.

Execute

Arithmetic or logical operation is performed.

Store

Result is stored back into memory.

---

6. Improving the Instruction Execution Cycle

Several techniques reduce execution time.

A. Instruction Pipelining

Multiple instructions execute simultaneously.

B. Cache Memory

Cache stores frequently accessed instructions.

Benefits:

- Faster instruction fetch
- Reduced RAM access
- Lower CPI

C. DMA (Direct Memory Access)

Normally,

I/O → CPU → Memory

DMA changes this to

I/O → Memory

CPU becomes free for processing.

Advantages

- Faster transfer
- Less CPU overhead
- Better real-time performance

D. Interrupt Prioritization

Instead of processing every interrupt equally,

High-priority events such as:

- Weapon detection
- Fire detection
- Accident detection

are processed first.

This reduces response time.

E. Multi-Core Processing

Different CPU cores process different camera feeds simultaneously.

Example:

Core	Assigned Camera
Core 1	Camera 1–50
Core 2	Camera 51–100
Core 3	Camera 101–150
Core 4	Camera 151–200

This significantly improves performance.

---

7. Additional Performance Enhancement Techniques

Technique	Purpose	Benefit
Cache Memory	Faster memory access	Reduced delay
DMA	Direct data transfer	Less CPU load
Multi-Bus	Parallel communication	Reduced congestion
Pipelining	Parallel instruction execution	Higher throughput
Multi-Core CPU	Parallel video processing	Faster analysis
Branch Prediction	Reduces pipeline stalls	Faster execution
Prefetching	Loads instructions in advance	Reduced waiting time

---

8. Overall Analysis

The delay in the surveillance system occurs because all three components—CPU, memory, and I/O—depend on one another. During peak hours, increased camera data leads to bus congestion, memory bottlenecks, and high CPU utilization. A **single-bus architecture** becomes a major limitation because every device shares the same communication path. Replacing it with a **multi-bus architecture** allows simultaneous data transfers, reducing waiting time and improving throughput. Additionally, techniques such as **instruction pipelining, cache memory, DMA, interrupt prioritization, and multi-core processors** optimize the instruction execution cycle and enable faster real-time threat detection.

Conclusion

A smart city surveillance system requires efficient coordination between the **CPU, memory, and I/O devices** to process continuous video streams without delay. During peak hours, performance issues arise mainly due to bus congestion, high CPU workload, and slow memory access. Adopting a **multi-bus architecture**, along with improvements such as **cache memory, pipelining, DMA, interrupt handling, and multi-core processing**, significantly enhances system performance. These architectural

Q2. Real-Time Stock Trading Application – CPI, Instruction Execution Cycle and CPU Performance

Introduction

A **real-time stock trading application** processes a large number of buy and sell orders every second. The application must execute transactions with **very low latency**, as even a delay of a few milliseconds can affect stock prices and trading outcomes.

The processor executes **millions of instructions per second (MIPS)**, but during peak trading hours the system experiences slow transaction processing. The major reason is a **high CPI (Clock Cycles Per Instruction)** caused by frequent memory accesses and branch instructions. Understanding the instruction execution cycle and adopting architectural improvements can significantly improve system performance.

---

1. Understanding CPU Performance

CPU performance depends on three factors:

- Clock Rate
- Instruction Count
- Clock Cycles Per Instruction (CPI)

The CPU performance equation is:

$$\text{CPU Execution Time} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

Example

Suppose:

- Instruction Count = **500 Million**
- CPI = **4**
- Clock Rate = **2 GHz**

$$Execution\ Time = \frac{500 \times 10^6 \times 4}{2 \times 10^9}$$
$$Execution\ Time = 1\ second$$

If CPI is reduced from **4 to 2**,

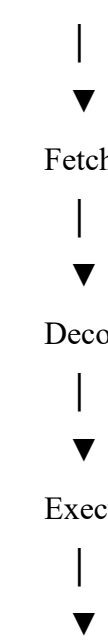
$$Execution\ Time = 0.5\ second$$

Thus, reducing CPI directly improves CPU performance.

---

2. Fetch–Decode–Execute Cycle

Every instruction executed by the processor passes through four stages.



Stage 1 – Fetch

The CPU fetches the next instruction from memory using the **Program Counter (PC)**.

During heavy trading:

- Millions of instructions are fetched.
- Frequent memory accesses increase fetch time.
- Cache misses force the CPU to wait for RAM.

Result:

- Increased instruction latency.

Stage 2 – Decode

The Control Unit interprets the instruction.

Example:

ADD R1,R2

The CPU identifies:

- Operation
- Source registers

Branch instructions also determine the next instruction address.

Incorrect branch prediction introduces delays.

Stage 3 – Execute

The ALU performs calculations.

Examples include:

- Price comparison
- Balance verification
- Risk calculation
- Tax computation

Complex arithmetic increases execution time.

Stage 4 – Write Back

The result is stored into registers or memory.

Example:

Updated Share Balance

Updated Account Balance

Transaction Status

Frequent memory writes increase CPI.

---

3. Causes of High CPI

The stock trading application executes millions of instructions, but several factors increase CPI.

A. Frequent Memory Access

Every transaction requires:

- Customer information
- Current stock price
- Order history
- Portfolio details

Continuous RAM access causes CPU stalls.

B. Branch Instructions

Trading systems frequently execute conditional statements.

Example

IF Balance > Share Price

    Buy Shares

ELSE

    Reject Transaction

Every branch changes program flow.

Incorrect branch prediction empties the pipeline and increases execution time.

C. Cache Misses

If required data is unavailable in cache,

CPU waits for RAM.

Since RAM is much slower than cache,

Execution time increases.

D. Pipeline Stalls

Dependency between instructions creates waiting periods.

Example

Read Balance

↓

Update Balance

↓

Store Balance

The next instruction waits until the previous one finishes.

E. Heavy Workload

During stock market opening or closing,

Thousands of users place orders simultaneously.

CPU utilization approaches 100%.

---

4. Impact of Fetch–Decode–Execute Cycle on Execution Time

Stage	Problem During High Load	Effect
Fetch	Memory delay	CPU waits for instructions
Decode	Frequent branch instructions	Longer decoding time
Execute	Complex calculations	Increased processing time
Write Back	Frequent memory updates	Higher memory traffic

Overall,

Each stage contributes to higher CPI and longer transaction processing time.

---

5. Architectural Enhancements to Reduce CPI

A. Cache Memory

Cache stores frequently used instructions and data close to the CPU.

CPU

Cache

RAM

Benefits

- Faster instruction fetch
- Reduced RAM access
- Lower CPI
- Improved throughput

B. Instruction Pipelining

Instead of executing one instruction at a time,

Multiple instructions execute simultaneously.

Pipeline Example

Clock Cycle	Instruction 1	Instruction 2	Instruction 3
1	Fetch		
2	Decode	Fetch	
3	Execute	Decode	Fetch
4	Write Back	Execute	Decode
5		Write Back	Execute

Benefits

- Higher instruction throughput
- Reduced execution time
- Better CPU utilization

C. Branch Prediction

The CPU predicts the outcome of branch instructions before execution.

Example

If Stock Price > Target

The processor predicts whether the condition is true or false and continues execution.

Advantages:

- Fewer pipeline stalls
- Faster execution
- Lower CPI

D. Out-of-Order Execution

Instead of waiting,

Independent instructions execute immediately.

Example

Instruction A → Waiting

Instruction B → Executed

Instruction C → Executed

Advantages

E. Superscalar Architecture

Modern processors execute multiple instructions in one clock cycle.

Example

Clock Cycle

ALU1 → Instruction 1

ALU2 → Instruction 2

FPU → Instruction 3

Benefits

- Higher Instructions Per Cycle (IPC)
- Lower CPI
- Faster transaction processing

F. Multi-Core Processors

Different cores process different users.

Example

CPU Core	Assigned Task
Core 1	Buy Orders
Core 2	Sell Orders
Core 3	Market Analysis
Core 4	Database Updates

Benefits

- Parallel processing
- Reduced response time
- Increased scalability

G. Prefetching

The processor loads future instructions before they are required.

6. Comparison of Architectural Enhancements

Technique	Purpose	Effect on CPI	Benefit
Cache Memory	Faster data access	Reduces	Faster memory access
Instruction Pipelining	Parallel instruction execution	Reduces	Higher throughput
Branch Prediction	Predicts branch outcome	Reduces	Fewer pipeline stalls
Out-of-Order Execution	Executes ready instructions	Reduces	Better CPU utilization
Superscalar Processing	Executes multiple instructions simultaneously	Reduces	Higher performance
Multi-Core Processor	Parallel task execution	Indirectly reduces	Faster processing
Prefetching	Loads instructions early	Reduces	Faster fetch cycle

---

7. Overall Analysis

The delays in the stock trading application occur because the processor spends extra clock cycles waiting for memory accesses and resolving branch instructions. These factors increase the **Clock Cycles Per Instruction (CPI)** and slow down transaction processing. Since every instruction passes through the **fetch, decode, execute, and write-back stages**, delays in any stage affect the overall execution time. By implementing **cache memory, instruction pipelining, branch prediction, out-of-order execution, superscalar architecture, prefetching, and multi-core processors**, the CPU can execute instructions more efficiently, reduce waiting time, and process a higher number of transactions per second.

Conclusion

A real-time stock trading system demands **high-speed and low-latency processing** to execute millions of transactions accurately. High CPI caused by frequent memory accesses and branch instructions significantly affects CPU performance and increases transaction delays. Optimizing the **fetch–decode–execute cycle** through advanced architectural enhancements such as **cache memory, pipelining, branch prediction, superscalar processing, out-of-order execution, prefetching, and multi-core processors** reduces CPI and improves overall system efficiency. These improvements enable faster transaction processing, higher throughput, and reliable performance during peak trading hours, making the system suitable for real-time financial applications.

Q3. Embedded System Based on 8085 Microprocessor – Addressing Modes and Instruction Set

Introduction

An **embedded system** is a dedicated computer system designed to perform a specific task. In an industrial temperature control unit, the **8085 microprocessor** continuously reads temperature values from sensors, processes the data, compares it with a predefined threshold, and controls actuators such as heaters, coolers, or alarms.

The accuracy of this system depends on the correct use of **addressing modes** and the **8085 instruction set**. Improper addressing may cause the processor to access incorrect memory locations or invalid data, resulting in inaccurate temperature readings and improper control actions. Therefore, selecting the appropriate addressing mode and instructions is essential for reliable system performance.

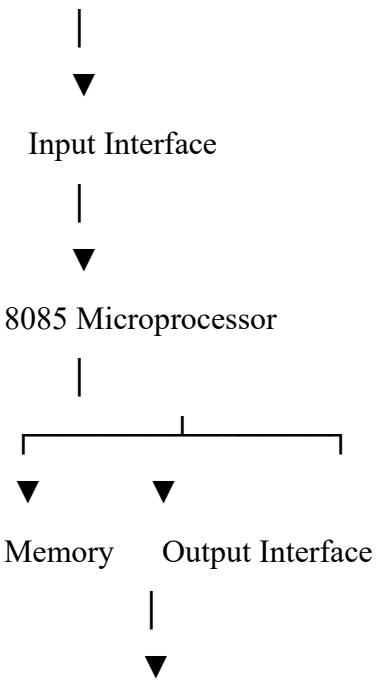
---

1. Working of the Embedded Temperature Control System

The temperature control system consists of the following components:

- Temperature Sensor
- 8085 Microprocessor
- Memory (RAM/ROM)
- Input/Output Interface
- Actuator (Fan/Heater)
- Display Unit

System Block Diagram



Heater / Fan / Alarm

Working Process

1. The temperature sensor measures the surrounding temperature.
2. The sensor sends the value to the 8085 through the input interface.
3. The processor stores the value in memory.
4. The temperature is compared with the preset threshold.
5. Based on the comparison, the processor activates the heater, fan, or alarm.

---

2. Addressing Modes in 8085

An **addressing mode** specifies how the processor accesses data or operands required for executing an instruction.

The 8085 supports five main addressing modes:

1. Immediate Addressing Mode
2. Register Addressing Mode
3. Direct Addressing Mode
4. Register Indirect Addressing Mode
5. Implied Addressing Mode

A. Immediate Addressing Mode

In this mode, the data is directly included in the instruction.

Example

MVI A,35H

Meaning:

- Load **35H** directly into register A.

Advantages

- Fast execution
- Simple programming
- No additional memory access

Disadvantages

- Suitable only for fixed values
- Cannot read sensor data directly

B. Register Addressing Mode

The operand is stored in a CPU register.

Example

MOV A,B

Meaning:

- Copy the contents of register **B** into register **A**.

Advantages

- Very fast
- No memory access required
- Efficient for temporary storage

Disadvantages

- Limited by the number of registers

C. Direct Addressing Mode

The memory address is specified explicitly in the instruction.

Example

LDA 2500H

Meaning:

- Load the data stored at memory location **2500H** into register A.

Advantages

- Easy to understand
- Direct access to memory

Disadvantages

- Slower than register addressing
- Incorrect addresses can cause wrong temperature readings

D. Register Indirect Addressing Mode

The memory address is stored in a register pair (HL).

Example

MOV A,M

Meaning:

- Load into register A the data stored at the memory location pointed to by the HL register pair.

Advantages

- Flexible memory access
- Efficient for arrays and sensor data

Disadvantages

- Incorrect HL values lead to invalid memory access

E. Implied Addressing Mode

The operand is implied in the instruction.

Example

CMA

Meaning:

- Complement the contents of the accumulator.

Advantages

- Simple execution
- No operand required

Disadvantages

- Limited applications

---

3. Comparison of 8085 Addressing Modes

Addressing Mode	Example	Operand Location	Advantages	Disadvantages
Immediate	MVI A,35H	Inside instruction	Fast and simple	Fixed data only
Register	MOV A,B	CPU Registers	Very fast	Limited registers
Direct	LDA 2500H	Memory Address	Easy to access memory	Slower than registers
Register Indirect	MOV A,M	Memory via HL pair	Flexible memory access	Wrong HL causes errors
Implied	CMA	Accumulator	Simple execution	Limited use

---

4. Causes of Incorrect Temperature Readings

Incorrect readings may occur due to improper use of addressing modes and programming errors.

A. Incorrect Memory Address

If the program accesses the wrong memory location, invalid sensor values are read.

Example:

LDA 2600H

Instead of:

LDA 2500H

This results in incorrect temperature data.

B. Wrong Register Pair

If the HL register pair points to the wrong memory location:

MOV A,M

The processor reads incorrect data, leading to faulty temperature control.

C. Register Overwriting

Important registers may be accidentally overwritten before processing.

Example:

MOV B,A

MVI A,00H

The original sensor value is lost.

D. Noise or Invalid Sensor Input

Electrical interference or faulty sensors may produce incorrect values that the processor reads and processes.

E. Programming Errors

Logical mistakes in comparison instructions or branching may activate the wrong actuator.

Example:

CMP B

JC COOLER

5. Effective Use of the 8085 Instruction Set

The 8085 instruction set can be used to improve accuracy and reliability.

Data Transfer Instructions

These instructions move data between registers, memory, and I/O ports.

Examples:

MOV A,B

LDA 2500H

STA 2600H

Purpose:

- Read sensor values
- Store processed data
- Transfer information efficiently

Arithmetic Instructions

These instructions perform calculations.

Examples:

ADD B

SUB C

INR A

DCR B

Purpose:

- Temperature averaging
- Incrementing counters
- Threshold calculations

Logical Instructions

These instructions compare and manipulate data.

Examples:

CMP B

ANA B

ORA C

Purpose:

- Compare current temperature with threshold
- Check status flags
- Validate sensor data

Branching Instructions

These instructions control program flow.

Examples:

JZ

JNZ

JC

JMP

Purpose:

- Turn heater ON
- Activate cooling system
- Trigger alarms
- Repeat monitoring loop

Input/Output Instructions

Examples:

IN 01H

OUT 02H

Purpose:

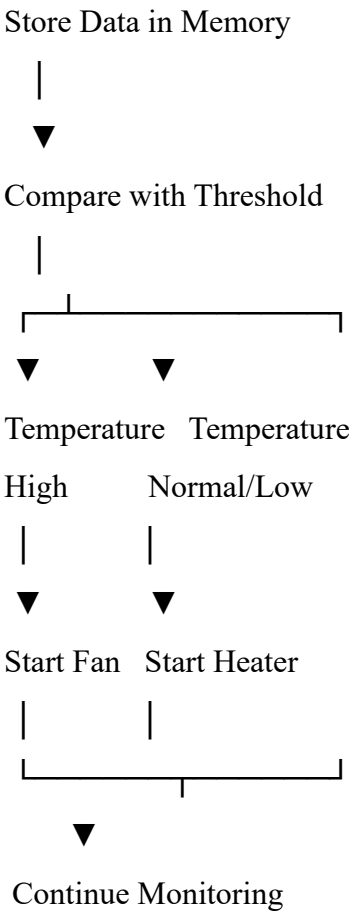
- Read temperature sensor values
- Send commands to actuators

---

6. Program Flow of Temperature Control

Start

Read Temperature Sensor



---

7. Methods to Improve Accuracy and Reliability

Technique	Purpose	Benefit
Correct Addressing Mode	Access correct memory location	Accurate sensor readings
Data Validation	Verify sensor values	Eliminates invalid data
Proper Register Management	Prevent register overwrite	Reliable execution
Error Checking	Detect abnormal readings	Improved safety
Interrupt Handling	Respond quickly to sensor changes	Real-time performance
Periodic Calibration	Maintain sensor accuracy	Reliable measurements
Efficient Instruction Set Usage	Optimize program execution	Faster and accurate processing

---

8. Overall Analysis

The performance of an **8085-based temperature control system** depends greatly on how the processor accesses data using addressing modes. Incorrect addressing can cause the CPU to read invalid memory locations, resulting in wrong temperature values and improper actuator control. By selecting suitable addressing modes, using the correct instruction set, validating sensor inputs, and implementing proper error checking, the system becomes more accurate, efficient, and reliable for industrial applications.

Conclusion

The **8085 microprocessor** plays a vital role in industrial temperature control systems by processing sensor data and controlling actuators. Proper use of **addressing modes** ensures that the processor accesses the correct data, while effective utilization of the **8085 instruction set** enables accurate data transfer, comparison, arithmetic operations, and control decisions. Techniques such as correct register management, data validation, interrupt handling, and periodic sensor calibration further improve system reliability. As a result, the embedded system can maintain precise temperature control, reduce processing errors, and ensure safe and efficient industrial operation.

Q4. 8086 Microprocessor – Segmented Memory, Stack Overflow and Addressing Modes

Introduction

The **8086 microprocessor** is a **16-bit processor** developed by Intel. One of its key features is **segmented memory architecture**, which allows the processor to efficiently manage memory and execute multiple programs. Instead of treating memory as one continuous block, the 8086 divides memory into different **segments**, making memory organization easier and improving program execution.

However, improper handling of these memory segments can lead to problems such as **incorrect data access**, **stack overflow**, and **program crashes**. Therefore, understanding segmented memory, instruction execution, and addressing modes is essential for ensuring reliable and efficient operation.

---

1. Segmented Memory Architecture in 8086

The **8086** has a **20-bit address bus**, allowing it to access **1 MB (2^{20} bytes)** of memory. Since it has only **16-bit registers**, it uses **segment registers** along with an **offset address** to generate a **20-bit physical address**.

Physical Address Formula

$$\boxed{\text{Physical Address} = (\text{Segment Address} \times 10H) + \text{Offset Address}}$$

For example:

- Code Segment (CS) = **2000H**
- Instruction Pointer (IP) = **1500H**

Physical Address:

$$(2000H \times 10H) + 1500H = 21500H$$

This mechanism allows the processor to efficiently utilize the entire 1 MB memory space.

---

2. Memory Segments in 8086

The 8086 divides memory into four logical segments.

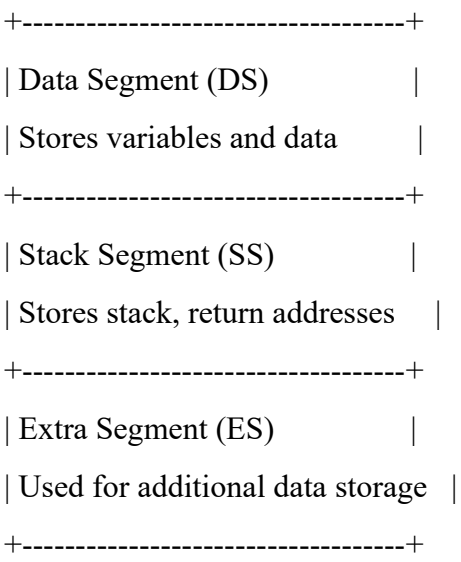
Memory Organization

1 MB Memory

+----------------------------------+

| Code Segment (CS) |

| Stores program instructions |



Each segment has a maximum size of **64 KB**.

A. Code Segment (CS)

The **Code Segment** stores the executable program instructions.

- Controlled by **CS (Code Segment Register)** and **IP (Instruction Pointer)**.
- CPU fetches instructions from this segment.

Example:

```
MOV AX, BX
ADD AX, CX
```

Advantages

- Organizes program instructions separately.
- Supports modular programming.
- Simplifies program execution.

B. Data Segment (DS)

The **Data Segment** stores variables, constants, and program data.

Example:

```
MOV AX, [2500H]
```

Purpose:

- Stores sensor values
- User inputs
- Arrays

C. Stack Segment (SS)

The **Stack Segment** stores:

- Return addresses
- Local variables
- Register contents
- Function parameters

The **Stack Pointer (SP)** points to the top of the stack.

Example:

PUSH AX

POP AX

The stack follows the **Last In First Out (LIFO)** principle.

D. Extra Segment (ES)

The **Extra Segment** provides additional storage.

It is mainly used in:

- String operations
- Data transfer
- Buffer storage

Example:

MOV ES, AX

---

3. Comparison of Memory Segments

Segment	Register Used	Purpose	Example
Code Segment	CS	Stores instructions	MOV AX, BX
Data Segment	DS	Stores variables	MOV AX,[2500H]
Stack Segment	SS	Stack operations	PUSH AX
Extra Segment	ES	Additional data storage	MOV ES,AX

---

4. Causes of Incorrect Data Access

Incorrect data access occurs when the processor accesses the wrong memory location.

A. Incorrect Segment Register

Example:

MOV DS, AX

If DS contains an incorrect value, all data accesses point to the wrong memory locations.

Result:

- Incorrect variables
- Invalid sensor values
- Wrong calculations

B. Incorrect Offset Address

Suppose:

Correct Offset:

2500H

Program uses:

2600H

The processor reads unintended memory contents, causing incorrect output.

C. Invalid Pointer Values

Registers like SI, DI, or BX are used as pointers.

If these registers contain incorrect addresses, the CPU fetches wrong data.

D. Memory Corruption

Improper writes to memory can overwrite important program data, leading to unexpected behavior.

---

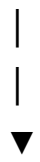
5. Stack Overflow in 8086

A **stack overflow** occurs when more data is pushed onto the stack than the allocated stack space can hold.

Stack Operation

Initial Stack

Top



PUSH AX

PUSH BX

PUSH CX

POP CX

POP BX

POP AX

The stack grows downward in memory.

Causes of Stack Overflow

- Excessive PUSH instructions
- Recursive function calls without termination
- Missing POP instructions
- Insufficient stack size
- Improper Stack Pointer (SP) management

Example:

PUSH AX

PUSH BX

PUSH CX

If POP instructions are not executed, the stack eventually overflows.

Effects of Stack Overflow

- Program crash
- Corrupted return addresses
- Incorrect function execution
- Data corruption
- System instability

---

6. Instruction Execution Cycle in 8086

Every instruction passes through four stages.

Instruction

Fetch

Decode

Execute

Store Result

Fetch

- CPU fetches instruction from the Code Segment using **CS:IP**.

Decode

- Control Unit interprets the instruction.

Execute

- ALU performs arithmetic or logical operations.

Store

- Result is written back to registers or memory.

Any error in segment registers or addressing during these stages can cause incorrect program execution.

---

7. Addressing Modes in 8086

The 8086 supports several addressing modes to access data efficiently.

A. Immediate Addressing

MOV AX,1234H

The operand is directly included in the instruction.

B. Register Addressing

MOV AX,BX

Data is transferred between CPU registers.

C. Direct Addressing

MOV AX,[2500H]

The instruction specifies the memory address explicitly.

D. Register Indirect Addressing

MOV AX,[BX]

The memory address is stored in a register.

E. Based Indexed Addressing

MOV AX,[BX+SI]

The effective address is calculated using two registers.

---

8. Comparison of Addressing Modes

Addressing Mode	Example	Advantage	Limitation
Immediate	MOV AX,1234H	Fast execution	Fixed data only
Register	MOV AX,BX	Very fast	Limited registers
Direct	MOV AX,[2500H]	Simple memory access	Slower than registers
Register Indirect	MOV AX,[BX]	Flexible	Requires correct pointer

Addressing Mode	Example	Advantage	Limitation
Based Indexed	MOV AX,[BX+SI]	Efficient for arrays	More complex calculations

---

9. Techniques to Ensure Correct Program Execution

Technique	Purpose	Benefit
Proper Segment Initialization	Set correct CS, DS, SS, ES values	Correct memory access
Balanced PUSH and POP Instructions	Prevent stack overflow	Stable execution
Proper Stack Pointer Management	Maintain stack integrity	Avoid crashes
Correct Addressing Mode Selection	Access correct data	Accurate execution
Pointer Validation	Prevent invalid memory access	Improved reliability
Error Handling	Detect memory errors	Increased system safety
Modular Programming	Separate code and data	Easier maintenance

---

10. Overall Analysis

The **8086 segmented memory architecture** improves memory organization by separating code, data, stack, and extra storage into independent segments. However, incorrect initialization of segment registers, invalid offset values, improper pointer management, or unbalanced stack operations can result in **incorrect data access** and **stack overflow**, causing unreliable program execution. Proper management of the **instruction execution cycle**, correct use of **addressing modes**, and disciplined handling of **PUSH/POP operations** ensure efficient and error-free execution of programs.

Conclusion

The **8086 microprocessor** uses **segmented memory architecture** to efficiently manage the 1 MB memory space by dividing it into **Code, Data, Stack, and Extra Segments**. Proper configuration of segment registers and addressing modes is essential to avoid incorrect data access, while balanced stack operations prevent stack overflow errors. By correctly managing the **instruction execution cycle**, selecting appropriate **addressing modes**, and maintaining proper stack and memory organization, the processor can execute multiple programs accurately, efficiently, and reliably. These practices improve system stability, enhance memory utilization, and ensure correct program execution in real-world applications.

Q5. Digital System – Number Systems, Data Representation, and CPU Performance

Introduction

Digital computers process all information using the **binary number system (0 and 1)** because electronic circuits operate using two voltage levels: **HIGH (1)** and **LOW (0)**. However, in real-world applications such as industrial automation, IoT, healthcare, and embedded systems, sensor data may be represented in **decimal**, **binary**, or **hexadecimal** formats.

If these values are not converted correctly, the processor performs incorrect calculations, leading to inaccurate outputs, system failures, and poor CPU performance. Therefore, understanding **number systems**, **data representation**, and **conversion techniques** is essential for reliable computer operation.

---

1. Importance of Number Systems

A **number system** is a method of representing numerical values using a specific base (radix).

The four commonly used number systems are:

- Binary (Base 2)
- Decimal (Base 10)
- Octal (Base 8)
- Hexadecimal (Base 16)

Each serves a different purpose in digital computing.

---

2. Types of Number Systems

A. Binary Number System (Base 2)

Binary consists of only **0 and 1**.

Example:

101101_2

Uses:

- CPU operations
- Memory storage
- Digital circuits
- Logic gates

Advantages

- Easy for electronic circuits.

- Reliable and error-resistant.
- Directly processed by the processor.

Disadvantages

- Long representation of large numbers.

B. Decimal Number System (Base 10)

Decimal uses digits **0–9**.

Example:

245_{10}

Uses:

- Human calculations
- User interfaces
- Business applications

C. Octal Number System (Base 8)

Octal uses digits **0–7**.

Example:

725_{8}

Uses:

- Compact representation of binary.
- Earlier computer systems.

D. Hexadecimal Number System (Base 16)

Hexadecimal uses:

0–9 and A–F

Example:

$2AF_{16}$

Uses:

- Memory addressing
- Machine language programming
- Debugging
- Embedded systems

---

3. Comparison of Number Systems

Number System	Base	Symbols Used	Common Applications
Binary	2	0,1	CPU operations, memory
Decimal	10	0–9	Human calculations
Octal	8	0–7	Compact binary representation
Hexadecimal	16	0–9, A–F	Memory addresses, machine code

---

4. Number System Conversion Methods

A. Decimal to Binary

Repeatedly divide the decimal number by 2 and note the remainders.

Example:

Convert $\mathbf{25_{10}}$ to Binary.

Division	Quotient	Remainder
25 ÷ 2	12	1
12 ÷ 2	6	0
6 ÷ 2	3	0
3 ÷ 2	1	1
1 ÷ 2	0	1

Reading remainders from bottom to top:

$25_{10} = 11001_2$

B. Binary to Decimal

Multiply each bit by its positional value.

Example:

101101_2

Bit	Positional Value	Product
1	32	32
0	16	0

Bit	Positional Value	Product
1	8	8
1	4	4
0	2	0
1	1	1

Total:

32+8+4+1=45

Therefore,

$101101_2 = 45_{10}$

C. Binary to Octal

Group binary digits into sets of **3 bits**.

Example:

111001_2

Grouping:

111 001

Conversion:

111 = 7

001 = 1

Answer:

$111001_2 = 71_8$

D. Binary to Hexadecimal

Group binary digits into **4 bits**.

Example:

10101111_2

Grouping:

1010 1111

Conversion:

1010 = A

1111 = F

Answer:

$10101111_2 = AF_{16}$

E. Hexadecimal to Binary

Replace each hexadecimal digit with its 4-bit binary equivalent.

Example:

$3C_{16}$

Hex	Binary
3	0011
C	1100

Answer:

$3C_{16} = 00111100_2$

---

5. Importance of Data Representation

Data representation determines how information is stored inside the computer.

Common representations include:

- Integers
- Floating-point numbers
- Characters (ASCII/Unicode)
- Signed and unsigned numbers

Correct representation ensures accurate processing and storage.

---

6. Causes of Incorrect Results

Incorrect number conversions may occur due to:

A. Wrong Base Conversion

Example:

Binary:

1010_2

Correct Decimal:

If interpreted incorrectly as decimal **1010**, calculations become incorrect.

B. Overflow

When data exceeds the storage capacity, overflow occurs.

Example:

An 8-bit register can store values only from **0 to 255**.

Storing **300** causes overflow and incorrect results.

C. Incorrect Hexadecimal Conversion

Example:

FF_{16}

Correct Decimal:

255

Incorrect conversion produces wrong memory values.

D. Data Truncation

Removing significant bits during conversion causes loss of information and inaccurate processing.

---

7. Impact on CPU Performance

Incorrect data representation affects CPU performance in several ways:

- Increased processing time due to repeated calculations.
- More exception handling and error correction.
- Incorrect memory addressing.
- Higher instruction count.
- Reduced execution efficiency.

These issues lower overall system performance.

---

8. Improving CPU Performance and System Reliability

Several techniques improve accuracy and efficiency.

A. Accurate Number Conversion

Always verify conversions before processing.

Benefit:

- Eliminates computational errors.

B. Proper Data Representation

Use appropriate data types for different applications.

Example:

- Integer for counts.
- Floating-point for temperature values.

Benefit:

- Accurate calculations.
- Reduced storage errors.

C. Error Detection

Use techniques such as:

- Parity bits
- Checksum
- CRC (Cyclic Redundancy Check)

Benefit:

- Detects transmission and storage errors.

D. Efficient Memory Management

Store data using the appropriate number format.

Benefit:

- Faster memory access.
- Lower CPU workload.

E. Data Validation

Validate sensor values before processing.

Example:

Temperature should remain within an expected range.

Benefit:

- Prevents invalid data from entering the system.

---

9. Techniques for Improving Reliability

Technique	Purpose	Benefit
Accurate Number Conversion	Prevent conversion errors	Correct calculations
Proper Data Representation	Store data correctly	Reliable processing
Error Detection (CRC/Parity)	Detect transmission errors	Improved accuracy
Data Validation	Verify sensor inputs	Eliminates invalid readings
Efficient Memory Management	Organize memory usage	Faster CPU execution
Overflow Checking	Prevent arithmetic overflow	Stable system performance
Appropriate Data Types	Match data to application	Better memory utilization

---

10. Overall Analysis

Digital systems depend on **accurate number systems and data representation** for correct processing of sensor data. Incorrect conversions between binary, decimal, octal, and hexadecimal can lead to computational errors, invalid memory addresses, and unreliable system behavior. Proper conversion techniques, suitable data representation, error detection mechanisms, and validation processes improve CPU efficiency and ensure dependable operation. These practices reduce processing delays, prevent errors, and enhance overall system performance and reliability.

Conclusion

Number systems and data representation form the foundation of digital computing. Since processors operate internally in **binary**, all external data must be converted accurately into the required format before processing. Correct conversion between **binary, decimal, octal, and hexadecimal** ensures accurate arithmetic operations, proper memory addressing, and efficient instruction execution. Furthermore, using suitable data representation, error detection methods, overflow checking, and data validation improves CPU performance, enhances system reliability, and minimizes processing errors. These practices are essential for developing robust and efficient digital systems in modern computing applications.